

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 14
SORTING



DISUSUN OLEH:

Rasya Putra Wibowo

109082500132

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

DASAR TEORI

Sorting atau pengurutan adalah proses menyusun data berdasarkan aturan tertentu agar lebih mudah diolah dan dianalisis. Pengurutan dapat dilakukan secara menaik (ascending) maupun menurun (descending) sesuai kebutuhan permasalahan. Dalam pemrograman, proses sorting sering digunakan untuk menyajikan data dalam bentuk yang lebih terstruktur dan mempermudah proses pencarian maupun pengambilan keputusan.

Salah satu algoritma pengurutan yang sederhana adalah Selection Sort. Algoritma ini bekerja dengan mencari elemen terkecil atau terbesar dari bagian data yang belum terurut, kemudian menukarnya dengan elemen pada posisi yang sesuai. Proses tersebut dilakukan berulang hingga seluruh elemen berada pada urutan yang benar. Karena mekanismenya sederhana, Selection Sort sering digunakan untuk mempelajari konsep dasar pengurutan data.

Pada permasalahan ini, data dibagi menjadi dua kelompok berdasarkan paritas bilangan, yaitu bilangan ganjil dan bilangan genap. Setelah dipisahkan, kelompok ganjil diurutkan secara menaik sedangkan kelompok genap diurutkan secara menurun. Teknik ini menunjukkan bahwa algoritma sorting dapat dikombinasikan dengan proses klasifikasi data untuk menghasilkan susunan keluaran yang sesuai dengan kebutuhan tertentu.

SOAL LATIHAN SELECTION SORT

1. Kerabat

Source Code:

```
package main

import "fmt"

const NMAX = 1000

func selectionSort(A []int, n int) {
    var i, j, idxMin, temp int

    for i = 0; i < n-1; i++ {
        idxMin = i

        for j = i + 1; j < n; j++ {
            if A[j] < A[idxMin] {
                idxMin = j
            }
        }

        temp = A[i]
        A[i] = A[idxMin]
        A[idxMin] = temp
    }
}

func main() {
    var n int
    var m int
    var i, j int
    var rumah [1000]int

    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Scan(&m)

        for j = 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSort(rumah[:], m)

        for j = 0; j < m; j++ {
            fmt.Print(rumah[j])

            if j < m-1 {
                fmt.Print(" ")
            }
        }
        fmt.Println()
    }
}
```

```
}  
}
```

Output:

```
func selectionSort(A []int, n int) {  
23 }  
24  
25 func main() {  
26     var n int  
27     var m int  
28     var i, j int  
29     var rumah [1000]int  
30  
31     fmt.Scan(&n)  
32  
33     for i = 0; i < n; i++ {  
34         fmt.Scan(&m)  
35  
36         for j = 0; j < m; j++ {  
37             fmt.Scan(&rumah[j])  
38         }  
39  
40         selectionSort(rumah[:], m)  
41     }  
42 }
```

PROBLEMS OUTPUT TERMINAL PORTS

> ▼ TERMINAL

```
PS C:\Users\ASUS\Downloads\Modul 14_Rasya Putra Wibowo> go run kerabat.go  
3  
5 2 1 7 9 13  
1 2 7 9 13  
6 189 15 27 39 75 133  
15 27 39 75 133 189  
3 4 9 1  
1 4 9
```

Deskripsi Program:

Program membaca banyak daerah kerabat Hercules sebanyak n . Untuk setiap daerah, program membaca jumlah rumah kerabat m beserta nomor rumahnya dan menyimpannya ke dalam array. Selanjutnya data diurutkan menggunakan algoritma Selection Sort, yaitu dengan mencari elemen terkecil dari bagian array yang belum terurut kemudian menukarnya dengan elemen pada posisi saat ini. Proses tersebut dilakukan berulang hingga seluruh data tersusun secara terurut membesar. Setelah pengurutan selesai, nomor rumah pada setiap daerah dicetak dalam satu baris sesuai urutan menaik.

2. Kerabat Dekat

Source Code:

```
package main

import "fmt"

func selectionSortAsc(A []int, n int) {
    var i, j, idxMin, temp int

    for i = 0; i < n-1; i++ {
        idxMin = i

        for j = i + 1; j < n; j++ {
            if A[j] < A[idxMin] {
                idxMin = j
            }
        }

        temp = A[i]
        A[i] = A[idxMin]
        A[idxMin] = temp
    }
}

func selectionSortDesc(A []int, n int) {
    var i, j, idxMax, temp int

    for i = 0; i < n-1; i++ {
        idxMax = i

        for j = i + 1; j < n; j++ {
            if A[j] > A[idxMax] {
                idxMax = j
            }
        }

        temp = A[i]
        A[i] = A[idxMax]
        A[idxMax] = temp
    }
}

func main() {
    var n, m int
    var i, j int
    var rumah [1000]int
    var ganjil [1000]int
    var genap [1000]int
    var jmlGanjil, jmlGenap int

    fmt.Scan(&n)

    for i = 0; i < n; i++ {
```

```

        fmt.Scan(&m)

        jmlGanjil = 0
        jmlGenap = 0

        for j = 0; j < m; j++ {
            fmt.Scan(&rumah[j])

            if rumah[j]%2 == 1 {
                ganjil[jmlGanjil] = rumah[j]
                jmlGanjil++
            } else {
                genap[jmlGenap] = rumah[j]
                jmlGenap++
            }
        }

        selectionSortAsc(ganjil[:], jmlGanjil)
        selectionSortDesc(genap[:], jmlGenap)

        for j = 0; j < jmlGanjil; j++ {
            fmt.Print(ganjil[j], " ")
        }

        for j = 0; j < jmlGenap; j++ {
            fmt.Print(genap[j])

            if j < jmlGenap-1 {
                fmt.Print(" ")
            }
        }

        fmt.Println()
    }
}-

```

Output:

```
kerabatDekat.go > ...
1  package main
2
3  import "fmt"
4
5  func selectionSortAsc(A []int, n int) {
6      var i, j, idxMin, temp int
7
8      for i = 0; i < n-1; i++ {
9          idxMin = i
10
11          for j = i + 1; j < n; j++ {
12              if A[j] < A[idxMin] {
13                  idxMin = j
14              }
15          }
16      }
17  }
```

PROBLEMS 2 OUTPUT TERMINAL PORTS

> TERMINAL

```
PS C:\Users\ASUS\Downloads\Modul 14_Rasya Putra Wibowo> go run kerabatDekat.go
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
```

Deskripsi Program:

Program membaca jumlah daerah kemudian data nomor rumah kerabat pada setiap daerah. Setiap nomor rumah dipisahkan ke dalam dua kelompok, yaitu kelompok rumah bernomor ganjil dan kelompok rumah bernomor genap. Data ganjil kemudian diurutkan menggunakan Selection Sort secara menaik (ascending), sedangkan data genap diurutkan menggunakan Selection Sort secara menurun (descending). Setelah proses pengurutan selesai, seluruh nomor rumah ganjil dicetak terlebih dahulu kemudian diikuti seluruh nomor rumah genap sehingga Hercules dapat mengunjungi rumah di satu sisi jalan terlebih dahulu sebelum berpindah ke sisi lainnya.

3. Median

Source Code:

```
package main

import "fmt"

const NMAX = 1000000

func insertionSort(A []int, n int) {
    var i, pass, temp int

    for pass = 1; pass < n; pass++ {
        temp = A[pass]
        i = pass - 1

        for i >= 0 && A[i] > temp {
            A[i+1] = A[i]
            i--
        }

        A[i+1] = temp
    }
}

func median(A []int, n int) int {
    if n%2 == 1 {
        return A[n/2]
    }

    return (A[(n/2)-1] + A[n/2]) / 2
}

func main() {
    var data [NMAX]int
    var n int
    var x int
    var hasil int

    n = 0

    for {
        fmt.Scan(&x)

        if x == -5313 {
            break
        }

        if x == 0 {
            insertionSort(data[:], n)
            hasil = median(data[:], n)
            fmt.Println(hasil)
        } else {
            data[n] = x
        }
    }
}
```



```
        n++  
    }  
}  
}
```

Output:

```
median.go > ...  
1  package main  
2  
3  import "fmt"  
4  
5  const NMAX = 1000000  
6  
7  func insertionSort(A []int, n int) {  
8      var i, pass, temp int  
9  
10     for pass = 1; pass < n; pass++ {  
11         temp = A[pass]  
12         i = pass - 1  
13  
14         for i >= 0 && A[i] > temp {  
15             A[i+1] = A[i]  
16             i--  
17         }  
18         A[i+1] = temp  
19     }  
20 }  
21  
22 func main() {  
23     var A []int  
24     var n int  
25     for {  
26         var x int  
27         fmt.Scan(&x)  
28         if x == -5313 {  
29             break  
30         }  
31         A = append(A, x)  
32     }  
33     insertionSort(A, len(A))  
34     fmt.Println(A)  
35 }
```

PROBLEMS 6 OUTPUT TERMINAL PORTS

> ▼ TERMINAL

PS C:\Users\ASUS\Downloads\Modul 14_Rasya Putra Wibowo> go run median.go
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12

Deskripsi Program:

Program membaca bilangan satu per satu hingga menemukan penanda akhir -5313. Setiap bilangan selain 0 dan -5313 disimpan ke dalam array. Ketika ditemukan angka 0, seluruh data yang telah tersimpan diurutkan menggunakan algoritma Insertion Sort, yaitu dengan menyisipkan setiap elemen ke posisi yang tepat pada bagian array yang sudah terurut. Setelah data terurut, median dihitung. Jika jumlah data ganjil, median adalah elemen tengah, sedangkan jika jumlah data genap, median adalah rata-rata dua elemen tengah yang dibulatkan ke bawah karena menggunakan pembagian bilangan bulat. Nilai median kemudian dicetak dan proses pembacaan data dilanjutkan hingga ditemukan -5313.

SOAL LATIHAN INSERTION SORT

1. Jarak

Source Code:

```
package main

import "fmt"

const NMAX = 1000

func insertionSort(A []int, n int) {
    var pass, i, temp int

    for pass = 1; pass < n; pass++ {
        temp = A[pass]
        i = pass - 1

        for i >= 0 && A[i] > temp {
            A[i+1] = A[i]
            i--
        }

        A[i+1] = temp
    }
}

func main() {
    var data [NMAX]int
    var n int
    var x int
    var jarak int
    var tetap bool
    var i int

    n = 0

    for {
        fmt.Scan(&x)

        if x < 0 {
            break
        }

        data[n] = x
        n++
    }

    insertionSort(data[:], n)

    for i = 0; i < n; i++ {
        fmt.Print(data[i])

        if i < n-1 {
```

```

        fmt.Print(" ")
    }
}
fmt.Println()

tetap = true

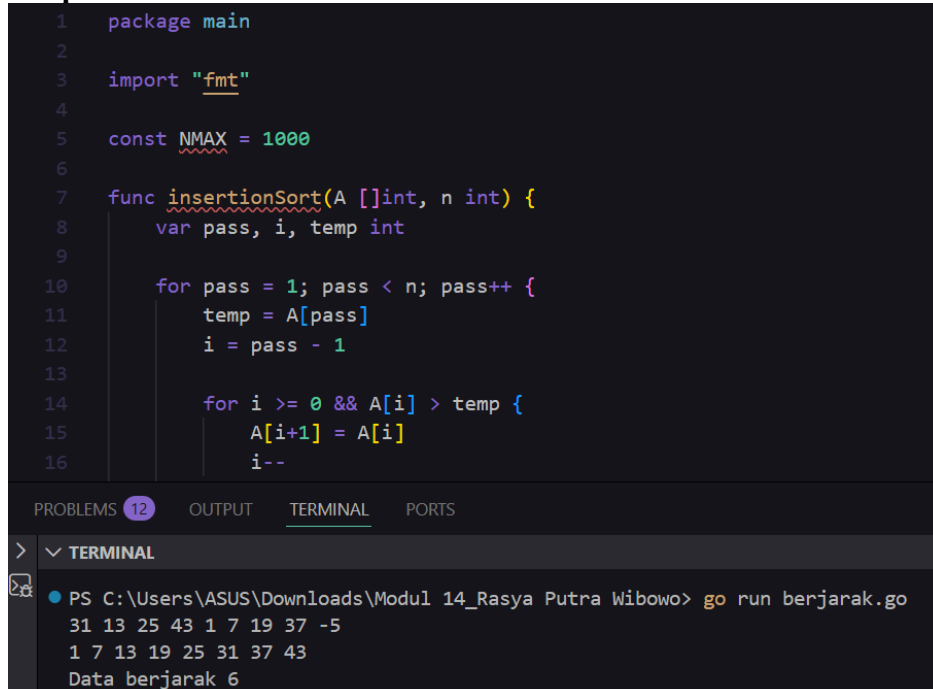
if n > 1 {
    jarak = data[1] - data[0]

    for i = 2; i < n; i++ {
        if data[i]-data[i-1] != jarak {
            tetap = false
        }
    }

    if tetap {
        fmt.Println("Data berjarak", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
} else {
    fmt.Println("Data berjarak 0")
}
}

```

Output:



The screenshot shows a Go IDE with a code editor and a terminal window. The code in the editor is a Go program that implements an insertion sort algorithm. The terminal window shows the output of the program, which is the sorted array and the distance between the first two elements.

```

1  package main
2
3  import "fmt"
4
5  const NMAX = 1000
6
7  func insertionSort(A []int, n int) {
8      var pass, i, temp int
9
10     for pass = 1; pass < n; pass++ {
11         temp = A[pass]
12         i = pass - 1
13
14         for i >= 0 && A[i] > temp {
15             A[i+1] = A[i]
16             i--

```

PROBLEMS 12 OUTPUT TERMINAL PORTS

> ▼ TERMINAL

```

● PS C:\Users\ASUS\Downloads\Modul 14_Rasya Putra Wibowo> go run berjarak.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6

```

Deskripsi Program:

Program membaca sekumpulan bilangan bulat dan berhenti ketika menemukan bilangan negatif. Hanya bilangan non-negatif yang disimpan ke dalam array. Setelah seluruh data dibaca, array diurutkan menggunakan algoritma Insertion Sort dengan cara menyisipkan setiap elemen ke posisi yang sesuai pada bagian array yang sudah terurut. Setelah pengurutan selesai, program memeriksa selisih antara setiap pasangan elemen yang berurutan. Jika seluruh selisih memiliki nilai yang sama, maka data dikatakan memiliki jarak tetap dan program menampilkan nilai jaraknya. Jika terdapat selisih yang berbeda, program menampilkan bahwa data berjarak tidak tetap.

2. Perpustakaan**Source Code:**

```
package main

import "fmt"

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    var i int

    fmt.Scan(n)

    for i = 0; i < *n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit,
            &pustaka[i].eksemplar,
            &pustaka[i].tahun,
            &pustaka[i].rating,
        )
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    var iMaks, i int

    iMaks = 0
```

```

        for i = 1; i < n; i++ {
            if pustaka[i].rating > pustaka[iMaks].rating {
                iMaks = i
            }
        }

        fmt.Println(
            pustaka[iMaks].judul,
            pustaka[iMaks].penulis,
            pustaka[iMaks].penerbit,
            pustaka[iMaks].tahun,
        )
    }

func UrutBuku(pustaka *DaftarBuku, n int) {
    var pass, i int
    var temp Buku

    for pass = 1; pass < n; pass++ {
        temp = pustaka[pass]
        i = pass - 1

        for i >= 0 && pustaka[i].rating < temp.rating {
            pustaka[i+1] = pustaka[i]
            i--
        }

        pustaka[i+1] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    var i, batas int

    if n < 5 {
        batas = n
    } else {
        batas = 5
    }

    for i = 0; i < batas; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n, r int) {
    var kiri, kanan, tengah int

    kiri = 0
    kanan = n - 1

    for kiri <= kanan {
        tengah = (kiri + kanan) / 2

        if pustaka[tengah].rating == r {

```

```

        fmt.Println(
            pustaka[tengah].judul,
            pustaka[tengah].penulis,
            pustaka[tengah].penerbit,
            pustaka[tengah].tahun,
            pustaka[tengah].eksemplar,
            pustaka[tengah].rating,
        )
        return
    } else if pustaka[tengah].rating < r {
        kanan = tengah - 1
    } else {
        kiri = tengah + 1
    }
}

fmt.Println("Tidak ada buku dengan rating seperti itu")
}

func main() {
    var pustaka DaftarBuku
    var n int
    var ratingCari int

    DaftarkanBuku(&pustaka, &n)

    CetakTerfavorit(pustaka, n)

    UrutBuku(&pustaka, n)

    Cetak5Terbaru(pustaka, n)

    fmt.Scan(&ratingCari)

    CariBuku(pustaka, n, ratingCari)
}

```

Output:

```
perpustakaan.go > ...
32 func CetakTerfavorit(pustaka DaftarBuku, n int) {
44     pustaka[iMaks].judul,
45     pustaka[iMaks].penulis,
46     pustaka[iMaks].penerbit,
47     pustaka[iMaks].tahun,
48 }
49 }
50
51 func UrutBuku(pustaka *DaftarBuku, n int) {
52     var pass, i int
53     var temp Buku
54
55     for pass = 1; pass < n; pass++ {
56         temp = pustaka[pass]
57         i = pass - 1
58     }
59 }

PROBLEMS 14 OUTPUT TERMINAL PORTS
> TERMINAL
PS C:\Users\ASUS\Downloads\Modul 14_Rasya Putra Wibowo> go run berjarak.go
PS C:\Users\ASUS\Downloads\Modul 14_Rasya Putra Wibowo> go run perpustakaan.go
6
B001 LaskarPelangi AndreaHirata Bentang 20 2005 95
B002 Bumi TereLiye Gramedia 15 2014 90
B003 Dilan PidiBaiq PastelBooks 12 2014 88
B004 FilosofiTeras HenryManampiring Kompas 10 2018 92
B005 AtomicHabits JamesClear Avery 8 2018 98
B006 CleanCode RobertMartin PrenticeHall 5 2008 96
AtomicHabits JamesClear Avery 2018
AtomicHabits
CleanCode
LaskarPelangi
FilosofiTeras
Bumi
92
FilosofiTeras HenryManampiring Kompas 2018 10 92
```

Deskripsi Program:

Program menggunakan tipe data struct Buku untuk menyimpan informasi setiap buku. Data buku dibaca melalui prosedur DaftarkanBuku, kemudian prosedur CetakTerfavorit mencari buku dengan rating tertinggi dan menampilkannya. Setelah itu data diurutkan berdasarkan rating secara menurun menggunakan algoritma Insertion Sort pada prosedur UrutBuku. Prosedur Cetak5Terbaru menampilkan maksimal lima judul buku dengan rating tertinggi. Terakhir, fungsi CariBuku menggunakan Binary Search pada data yang sudah terurut untuk mencari buku dengan rating tertentu. Jika ditemukan, seluruh informasi buku ditampilkan, sedangkan jika tidak ditemukan program mencetak pesan bahwa tidak ada buku dengan rating tersebut.

DAFTAR PUSTAKA

- Kadir, Abdul. 2018. *Dasar Pemrograman Go*. Yogyakarta: Andi Publisher.
- Wirth, Niklaus. 1986. *Algorithms + Data Structures = Programs*. New Jersey: Prentice Hall.
- Aho, Alfred V., Hopcroft, John E., dan Ullman, Jeffrey D. 1983. *Data Structures and Algorithms*. Boston: Addison-Wesley.